

Task 1. 类、对象与构造方法

1. 请说明你的构造方法是否用到了 this，如果用了，它在这里起什么作用。
 - 用到了,我写了一个**有参构造器**,其中**成员变量**与传入的变量同名了,会出现变量传不到**对象**的情况,所以需要 this 来**指向对象**
2. 请用自己的话解释“类”和“对象”的关系。
 - **类**是一个模版,类似于一个**空表格**,规定了在这个类中的**对象**能做什么,调用什么**方法**
 - **对象**是一个具体的**实例**,类似于一个人按照**类**给出的表格填入了**具体信息**
3. **附注**:为了在**main方法**中调用person的方法我把private的前缀修改成public了(虽然题目没有要求,我想试试构造器是否能运行)
我写了一个**Test类**用来运行**main方法**
4. 代码如下:

```
package com.Example.personDemo;

public class person {
    private String name;
    private int age;
    private int sex;

    public person(){

    }

    public person(String name,int age,char sex){
        this.name = name;
        this.age = age;
        this.sex = sex;
    }

    public person(person p){
        this.name = p.name;
        this.age = p.age;
        this.sex = p.sex;
    }

    public void eat() {
        System.out.println(name + "正在吃东西");
    }

    public void sleep() {
        System.out.println(name + "正在睡觉");
    }

    public void dadoudou() {
        System.out.println(name + "正在打豆豆");
    }
}
//这是Test类
package com.Example.personDemo;
```

```
public class Test {
    public static void main(String[] args){
        person s = new person("小明",18,'男');
        s.dadoudou();
    }
}
```

Task 2. 访问修饰符、实例成员与静态成员

1. 尝试在以下位置访问它们，并总结访问范围：
 - 当前类内部:不论什么访问修饰符都可以访问
 - 同一个包下的其他类:只有**private**修饰符
 - 不同包下的类:**protect**在子类不同包可以访问,**public**在所有类下都可访问
2. 说明 public、private、protected、默认访问权限分别代表什么。
 - public:所有类均可访问
 - protect:在**无关其他包**内不能访问
 - 默认访问权限:在同一个包内可以访问
 - private:**仅可以在本类**访问
3. 为什么静态方法可以直接通过类名调用
 - 静态方法属于**这个类**不需要**依赖对象**来调用
4. 为什么实例方法通常需要先创建对象
 - 实例方法**属于对象**,方法一般先读取**实例变量**,其中**实例变量**存在对象中
5. 修改后的代码:

```
//变量全部公开
public String name;
public int age;
public int sex;

//创建一个静态变量
static int count = 0;

public person(){

}

//增加了一个显示人数的静态方法
public static void showCount() {
    System.out.println("当前人数为: " + count);
}
```

Task 3. 继承与代码复用

```
//Penguin子类
public class Penguin extends Animal {
    public Penguin(String myName, int myId) {
        super(myName, myId);
    }

    @Override
    public void introduction() {
```

```

        System.out.println("大家好！我是企鹅" + id + "号" + name + "，泥虫Q币嘛。");
    }
}
//Mouse子类
public class Mouse extends Animal {
    public Mouse(String myName, int myId) {
        super(myName, myId);
    }
    @Override
    public void introduction() {
        System.out.println("大家好！我是老鼠" + id + "号" + name + "，吱吱吱。");
    }
}
}

```

6. 解释：为什么Java 不支持类的多继承？

- 因为当一个子类继承了两个拥有同名方法的父类时,调用方法时就不知道执行哪个

Task 4. 多态与抽象设计

1. 我选择抽象类方案

```

//父类Shape
public abstract class Shape {
    public abstract double calPerimeter();
    public abstract double calArea();
}

//子类Circle
public class Circle extends Shape {
    private double R;
    public Circle(double R){
        this.R = R;
    }
    @Override
    public double calArea(){
        return Math.PI * R * R;
    }
    public double calPerimeter(){
        return Math.PI * 2 * R;
    }
}

//子类Rectangle
public class Rectangle extends Shape{
    private double a,b;
    public Rectangle(double a,double b){
        this.a = a;
        this.b = b;
    }
    @Override
    public double calPerimeter(){
        return 2*(a+b);
    }
}

```

```

        public double calArea(){
            return a*b;
        }
    }

    //子类Triangle
    public class Triangle extends Shape {
        private int a,b,c;
        public Triangle(int a,int b,int c){
            this.a = a;
            this.b = b;
            this.c = c;
        }
        @Override
        public double calPerimeter(){
            return a+b+c;
        }
        @Override
        public double calArea(){
            double p = (a+b+c)/2;
            return Math.sqrt(p*(p-a)*(p-b)*(p-c));
        }
    }

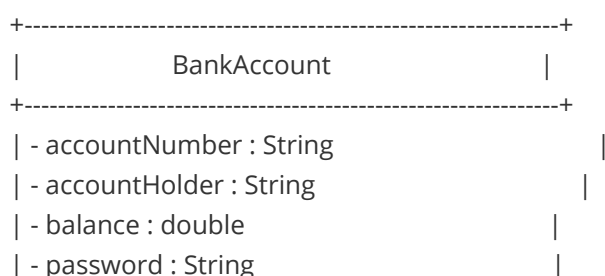
    //循环调用
    import java.util.ArrayList;
    public class Testloop {
        public static void main(String[] args){
            ArrayList<Shape> shapeList = new ArrayList<>();

            shapeList.add(new Circle(2));
            shapeList.add(new Rectangle(4, 6));
            shapeList.add(new Triangle(3, 4, 5));
            for(Shape shape : shapeList){
                System.out.println("面积:"+shape.calArea()+"周
长:"+shape.calPerimeter());
            }
        }
    }
}

```

Task 5. 封装与接口设计

- private 是封装的**核心手段**,使变量只能在**当前类内**被访问,必须额外设置一个 set 方法,因此可以在这个方法内增加**校验手段**
 - public 是令变量可以被外界随意访问,不利于**保护**变量(这里我注意到在Task2中我为了方便把成员变量都设置为public是错误的)
 - protect 多用于**继承**的时候,使变量仅被**父子类**访问
- 类图思路:



```

+-----+
| + BankAccount(accountNumber:String, accountHolder:String, initialBalance:double,
| password:String) |
| + deposit(amount:double):void |
| + withdraw(amount:double, inputPassword:String):boolean |
| + transfer(recipient:BankAccount, amount:double, inputPassword:String):boolean |
| + getBalance():double |
| + getAccountInfo():String |
| - validatePassword(inputPassword:String):boolean |
| - validateAmount(amount:double):boolean |
+-----+

```

3. 源代码:

```

package com.Bankaccount;
public class BankAccount {
    // TODO: 修改属性的可见性
    private String accountNumber;
    private String accountHolder;
    private double balance;
    private String password;

    BankAccount(String accountNumber, String accountHolder, double
initialBalance, String password) {
        // TODO
        //金额不能为0或负数
        if(initialBalance <= 0) {
            throw new IllegalArgumentException("Initial balance cannot be zero
or negative.");
        }
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
        this.password = password;
    }

    void deposit(double amount) {
        // TODO
        this.balance += amount;
    }

    boolean withdraw(double amount, String inputPassword) {
        // TODO
        if (validatePassword(inputPassword) && validateAmount(amount) &&
this.balance >= amount){
            this.balance -=amount;
            return true;
        }
        return false;
    }

    boolean transfer(BankAccount recipient, double amount, String inputPassword)
{
        // TODO
        if (this.withdraw(amount, inputPassword)) {
            recipient.deposit(amount);

```

```

        return true;
    }
    return false;
}

double getBalance() {
    // TODO
    return this.balance;
}

String getAccountInfo() {
    // TODO
    return "Account Number: " + this.accountNumber + ", Account Holder: " +
this.accountHolder;
}

// 只需修改可见性
private boolean validatePassword(String inputPassword) {
    return this.password.equals(inputPassword);
}

// 只需修改可见性
private boolean validateAmount(double amount) {
    return amount > 0;
}
}

```

4. 虽然题目要求 `validatePassword` 和 `validateAmount` 只需修改可见性,但是我还是修改了**验证的逻辑**
5. 这是我的**测试类**和**运行结果**:

```

public class Bankaccount_test {
    public static void main(String[] args) {
        BankAccount account1 = new BankAccount("123456", "帅哥", 5000, "123123");
        BankAccount account2 = new BankAccount("654321", "美女", 3000, "321321");
        account1.deposit(1000);
        System.out.println(account1.getAccountInfo());
        account1.withdraw(3000, "123123");
        System.out.println("Account 1 Balance: " + account1.getBalance());
        account1.transfer(account2, 2000, "123123");
        System.out.println("Account 1 Balance: " + account1.getBalance());
        System.out.println("Account 2 Balance: " + account2.getBalance());
    }
}

```

```

Account Number: 123456, Account Holder: 帅哥
Account 1 Balance: 3000.0
Account 1 Balance: 1000.0
Account 2 Balance: 5000.0

```